

Viajaí Bolso

Documentação Técnica e Funcional do Projeto

App de controle de gastos em viagens — Flutter

Projeto desenvolvido como trabalho de conclusão do 3º período do curso de Análise e Desenvolvimento de Sistemas, em fase de evolução para se tornar um aplicativo funcional completo.

Documento gerado e atualizado automaticamente a cada alteração relevante no código.

1. Visão Geral do Projeto

O **Viajaí Bolso** é um aplicativo mobile (Flutter) para controle financeiro de viagens: o usuário cadastra viagens — cada uma com destino, período e orçamento — e registra despesas associadas a cada viagem, categorizadas (alimentação, transporte, etc.), acompanhando o quanto já foi gasto em relação ao orçamento definido.

O projeto nasceu como protótipo de conclusão do 3º período do curso de Análise e Desenvolvimento de Sistemas e agora está em evolução para se tornar um aplicativo funcional completo, a ser apresentado como projeto final do curso. Este documento acompanha essa evolução: a cada mudança relevante no código, uma nova seção é adicionada ao **Histórico de Alterações** (seção 7), mantendo sempre a descrição das seções anteriores atualizada com o estado mais recente do app.

1.1 Principais dependências (pubspec.yaml)

Pacote	Finalidade
http	Comunicação com o backend via requisições REST (GET/POST/PUT/DELETE)
shared_preferences	Persistência local simples — usada para manter a sessão do usuário logado
image_picker / image_picker_for_web	Seleção de fotos (foto de perfil e foto da viagem), convertidas para base64
intl	Suporte a internacionalização/formatação (incluído, uso pontual)

1.2 Arquitetura em camadas

O código em `lib/` está organizado em três camadas, sem uso de gerenciador de estado externo (Provider, Riverpod, Bloc) — o controle de estado é feito manualmente com `setState`:

- **constants.dart** — configurações globais: endereço do backend e paleta de cores do app.
- **models/** — classes de dados puras (Despesa, Categoria, Usuario, Viagem), cada uma com método `fromJson` para converter a resposta da API.
- **services/** — camada de acesso ao backend via HTTP (AuthService, ViagemService, DespesaService, DashboardService).
- **screens/** — telas do app, que chamam os services diretamente (não há camada intermediária de controller/repository).

Ponto de atenção técnico: o endereço do backend (`baseUrl`) está fixo no código como um IP de rede local (`http://192.168.0.9:5000`), sem variável de ambiente para produção. Isso será um dos primeiros pontos a resolver na evolução do protótipo para app funcional.

2. Modelos de Dados (lib/models)

2.1 Despesa e Categoria — `despesa.dart`

Representa uma despesa individual vinculada a uma viagem e a uma categoria.

Campo (Despesa)	Tipo	Descrição
idDespesa	int?	Identificador único (gerado pelo backend)
descricao	String	Descrição da despesa
valor	double	Valor gasto
data / hora	String / String?	Data e hora do gasto
formaPagamento	String	Ex.: Cartão de crédito, Débito, Dinheiro, Pix, Outros
idViagem / idCategoria	int	Referências à viagem e categoria associadas
categoria / icone	String? / String?	Nome e ícone da categoria (retornados pelo backend)

A classe **Categoria** representa uma categoria de despesa (id, nome, ícone), obtida via API.

2.2 Usuario — `usuario.dart`

Campo	Tipo	Descrição
idUsuario	int	Identificador do usuário
nome / email	String	Dados básicos de cadastro
foto	String?	Foto de perfil (base64)
moedaPadrao	String	Moeda padrão do usuário (default: 'BRL')
token	String?	Token de sessão/autenticação

2.3 Viagem — `viagem.dart`

Campo	Tipo	Descrição
idViagem	int?	Identificador da viagem
nome / destino	String	Nome e destino da viagem
foto	String?	Foto da viagem (base64)
dataInicio / dataFim	String	Período da viagem (AAAA-MM-DD)
orcamento	double	Orçamento definido para a viagem
totalGasto / percentualGasto	double	Valores agregados calculados pelo backend
idUsuario	int?	Dono da viagem

3. Serviços — Comunicação com o Backend (lib/services)

Todos os serviços usam o pacote `http` para se comunicar com uma API REST externa (o código do backend não faz parte deste repositório Flutter). Não há banco de dados local para viagens/despesas — o app depende do backend estar acessível a cada tela.

3.1 AuthService — autenticação e perfil

Método	Endpoint	Função
<code>cadastrar(nome, email, senha)</code>	POST <code>/cadastro</code>	Cria uma nova conta
<code>login(email, senha)</code>	POST <code>/login</code>	Autentica o usuário
<code>logout(idUsuario)</code>	POST <code>/logout</code>	Encerra a sessão no backend
<code>recuperarSenha(email)</code>	POST <code>/recuperar-senha</code>	Envia código de recuperação por e-mail
<code>redefinirSenha(...)</code>	POST <code>/redefinir-senha</code>	Redefine a senha usando o código recebido
<code>getPerfil(idUsuario)</code>	GET <code>/perfil/{id}</code>	Busca os dados do perfil
<code>atualizarPerfil(...)</code>	PUT <code>/atualizarperfil</code>	Atualiza nome, e-mail, moeda e/ou foto
<code>alterarSenha(...)</code>	PUT <code>/alterarsenha</code>	Troca a senha do usuário

3.2 ViagemService, DespesaService e DashboardService

Classe / Método	Endpoint	Função
<code>ViagemService.listar(idUsuario)</code>	GET <code>/viagens/{id}</code>	Lista as viagens do usuário
<code>ViagemService.cadastrar / atualizar / remover</code>	POST/PUT/DELETE	CRUD de viagens
<code>DespesaService.listar(idViagem)</code>	GET <code>/despesas/{id}</code>	Lista despesas de uma viagem
<code>DespesaService.listarCategorias()</code>	GET <code>/categorias</code>	Lista categorias disponíveis
<code>DespesaService.cadastrar / atualizar / remover</code>	POST/PUT/DELETE	CRUD de despesas
<code>DashboardService.getDashboard(idUsuario)</code>	GET <code>/dashboard/{id}</code>	Dados agregados: viagem ativa, gastos de hoje, maior gasto, últimas despesas

Observação técnica: nenhum serviço verifica o `statusCode` da resposta HTTP; o tratamento de erro é feito de forma genérica nas telas (mensagem fixa "Erro de conexão com o servidor"). Esse ponto é candidato a melhoria na próxima fase.

4. Telas e Funcionalidades (lib/screens)

SplashScreen

Tela inicial. Exibe a logo por 2 segundos e verifica se há uma sessão salva (SharedPreferences). Se houver, leva direto ao mainScreen; senão, à tela de Login.

LoginScreen

Login com e-mail e senha, opção de mostrar/ocultar senha, links para "Esqueci minha senha" e "Cadastre-se". Em caso de sucesso, salva a sessão localmente e abre o mainScreen.

CadastroScreen

Cadastro de novo usuário (nome, e-mail, senha, confirmação de senha) com aceite obrigatório dos Termos de Uso (texto exibido na própria tela).

RecuperarSenhaScreen

Fluxo de recuperação de senha em duas etapas: solicitar código por e-mail e, em seguida, informar o código recebido junto com a nova senha.

MainScreen

Shell de navegação principal, com barra inferior de 4 abas: Início (Dashboard), Viagens, Despesas e Perfil.

DashboardScreen

Tela inicial pós-login: saudação personalizada, resumo da viagem ativa (gasto vs. orçamento, com indicador circular de progresso), gastos do dia, maior gasto e lista das despesas mais recentes. Suporta atualizar puxando a tela (pull-to-refresh).

ViagensScreen

Lista todas as viagens cadastradas em cards (foto, nome, destino, datas, progresso do orçamento), com opções de editar e excluir cada viagem, e botão para cadastrar uma nova.

NovaViagemScreen

Formulário de criação/edição de viagem: foto (câmera/galeria), nome, destino, datas de início e fim, e orçamento.

DespesasScreen

Seleciona uma viagem e lista suas despesas, com filtro por categoria. Permite cadastrar, editar e excluir despesas por um formulário em painel deslizante, incluindo categoria, descrição, valor, data e forma de pagamento. Mostra o total das despesas filtradas.

PerfilScreen

Exibe e permite editar os dados do usuário (foto, nome, e-mail), alterar a senha e encerrar a sessão (logout), limpando os dados salvos localmente.

5. Fluxo de Navegação

Resumo do fluxo entre as telas do aplicativo, a partir da inicialização:

- **Splash** → verifica sessão salva → **Login** (se não houver sessão) ou **MainScreen** (se já houver uma sessão salva).
- **Login** → sucesso leva ao **MainScreen**; "Cadastre-se" leva ao **Cadastro**; "Esqueci minha senha" leva à **Recuperação de Senha**.
- **MainScreen** organiza 4 abas fixas: **Dashboard** | **Viagens** (→ Nova/Editar Viagem) | **Despesas** (formulário em painel deslizante) | **Perfil**.
- **Perfil** → **Sair da conta** encerra a sessão e retorna à tela de **Login**, limpando todo o histórico de navegação.

6. Estado Atual e Pontos de Evolução

Como protótipo acadêmico, o app já demonstra o fluxo completo de cadastro, login, gestão de viagens e despesas. Os pontos abaixo estão mapeados como próximos passos para transformá-lo em um app funcional completo — úteis tanto para o planejamento das próximas entregas quanto para a banca de apresentação:

- Endereço do backend fixo no código (IP de rede local); precisa de configuração por ambiente (desenvolvimento/produção) e uso de HTTPS.
- Tratamento de erros de rede é genérico (sem checagem de status HTTP nem mensagens específicas por tipo de erro).
- Algumas opções da interface ainda são placeholders sem função: botão "Ver todas" no Dashboard, filtro de despesas, e os itens "Moeda padrão", "Configurações", "Sobre o app" e "Suporte" na tela de Perfil.
- Termos de Uso exibidos no Cadastro são apenas um texto estático, sem uma tela própria de conteúdo legal.
- Não há persistência local (offline) dos dados de viagens/despesas — o app depende do backend estar sempre acessível.
- Os modelos de dados (Despesa, Usuario, Viagem) não possuem método toJson(); o envio de dados ao backend é remontado manualmente em cada tela.
- Não há rotas nomeadas nem testes automatizados no projeto até o momento.

7. Histórico de Alterações

Esta seção registra, em ordem cronológica, as mudanças feitas no projeto a partir desta documentação — servindo como registro de evolução para a apresentação final.

Versão 1 — Estado inicial do protótipo (baseline)

Data: 27/08/2026

Documentação inicial gerada a partir do código-fonte existente do projeto (entregue ao final do 3º período), antes do início de novas implementações. Estabelece o ponto de partida do histórico de mudanças que será usado para a apresentação final do curso.

- Levantamento completo da arquitetura atual (models, services, screens)
- Nenhuma alteração de código foi feita nesta etapa — apenas documentação do estado existente